问题分析. 既然传感器在，就怀疑速率或寄存器写入顺序：依次用 100、50、25、10 kHz 重读芯片 ID，每档都按 Linux 的比例同时更新 BSC_DIV 和边沿采样延迟 BSC_DEL（不继承固件留下的值），并在写完地址、长度、FIFO 后加 dsb sy，保证配置在 START 之前到达控制器。四档全部是 ERR，速率和 MMIO 写序都被排除了；剩下的只能是 BSC0 到 GPIO44/45 这条路由本身，或者固件对 BSC0 的占用。继续在这里花时间，收益已经很低。

解决方案. 绕开 BSC0：用 GPIO 完成完整的软件 SCCB 事务（写寄存器地址、STOP、START、读一个字节），读到 0x5647 后打印 `using GPIO software SCCB; BSC0 bypassed`，此后传感器的所有读写都走这条路。寄存器表里 0x0103=1 会让传感器软件复位、期间不应答，写完这一项等 10 ms；普通寄存器写最多重试三次。软件 SCCB 慢，但只用来写一百来个寄存器，图像走 CSI-2，不受影响。

问题 5：软件 SCCB 偶发读出 0x3631，随后 BUS-STUCK

日志输出.

```
ov5647: software SCCB chip-id=3631 attempt=1                    (格式)
...
camera: full GPIO I2C scan bus stuck at 0x..
```

问题分析. 地址被 ACK 了，读回的却是 0x3631——它不是任何传感器的 ID，事务层面成功、数据位却采错了，说明时序处于边缘。旧代码随后的两个动作让情况更糟：每次软件事务前无条件送 9 个 SCL 恢复脉冲；读到错误 ID 后立即扫描整个 7 位地址空间。对一个状态已经不稳的模块，这些都是额外的扰动，扫描最终把总线卡住（BUS-STUCK）。

解决方案. （gpio-sccb-v7）软件时钟降到约 25 kHz（半周期约 20 µs）；任何不是 0x5647 的完整读数都被拒绝重读（日志 `rejected unstable chip-id=..`）；恢复脉冲只在发现 SDA 被拉低时才送，最多 9 个，然后统一发 STOP；只要 0x36 已经完成过寄存器事务，就不再做全地址扫描，而是打印 `address 0x36 responds but chip-id is unstable; power-cycle retry required` 后直接断电重来；上电前 CAM_GPIO0 先拉低 20 ms 让模块放电，置高后等 100 ms 再访问。

问题 6：0x36 ACK，但三次读 ID 都失败

日志输出.

```
ov5647: software SCCB ID read retry 1/3 (probe=ACK)             (格式)
ov5647: software SCCB ID read retry 2/3 (probe=ACK)
ov5647: software SCCB ID read retry 3/3 (probe=ACK)
ov5647: address 0x36 ACK but chip-id read failed; power-cycle retry required (fsel=.. levels=..)
```

问题分析. v7 在真机上仍会出现这种情况。看调用顺序就明白了：每次上电，驱动仍然先用 BSC0 读 ID，失败几次之后才切到 GPIO。问题 4 已经证明这些 BSC 事务注定失败，而它们留下的半截事务会让传感器的 SCCB 接口进入不确定状态，紧接着的软件读取就跟着失败。

解决方案. （gpio-sccb-v8）从第一次读 0x300a/0x300b 起就直接用 25 kHz 软件 SCCB，BSC0 只初始化、不再发事务，日志变为 `probing with 25 kHz GPIO SCCB; BSC0 bypassed` 和 `software SCCB power-on chip-id=5647`。如果 0x36 已 ACK 但读寄存器仍失败，本轮立即断电，交给 `camera_bind()` 做下一次完整的电源周期（用户首次 `read()` 时最多三次，每次间隔 100 ms），不再探测其他地址。7.9.10 小节的日志就是 v8 的样子。